UNIT-2.

PAGE :
DATE : / /

Input Output Statement :-

C is a functional Programming language in which to perform these Input output operations. C Provid a library functions. This library is called in statement input output library. It is denoted by stdio.h. The header file containing such library functions is called stdio.h

Some of the standard input output functions in C are given below.

1. Scanf( ).
2. Printf( )
3. getchar( )
4. putchar( )
5. gets( )
6. puts( )
7. getch( ).

There are two types of Input output Statement which are as.

1. Formatted I/O Statements
2. Unformatted I/O Statements

Formatted Input Output statement enable the user to especific the type of data.

On the other hand the unformatted of input output Statement do not especify the type of data.

PAGE:
DATE: / /

Type of I/O Statement	Input Statement	Output Statement
Formatted	Scanf( )	printf(
Unformatted	getchar( )	putchar(
	gets( )	puts( )

☆. <u>Reading Input data</u> :-
Input data can be entered into the memory from a Standard Input device (Keyboard), C provide the Scanf( ) library functions for entering input data. This function can takes all type of value (Numric, character, String) as Input. The Scanf( ) functions can be written as. ~~Scanf("% Control String", & address 2, ----);~~
Scanf("% Control string", & address2,--);

This function should have atleast two parameters.
First Parameter is a control String
Second " is a Address of variable.

☆. 1. <u>Control string</u> :- Control string contains Conversion specification character. It should be within double quotes.

Each character group is a combination of the (%) Symbole and one of the conversion. This conversion Specification character may be one or more. It depend on the number of variables, we want to Input.

2. <u>Address of Variables</u> :-
The address of variables is found by preceding the variable name by an ampersand (&) sign. This Sign is called the address operator.
It gives the Starting address of the variable name in memory.
A String variable is not procede by ampersand (&) Sign to get the address.

☆. <u>Conversion Specification character</u> :-

%c → a single character
%d → a decimal integer
%f → a floating Point numbers
%lf → a long range of floating point no.s
%o → an octal Numbers.
%X → a hexadecimal no.
%s → String.

PAGE:
DATE: / /

4. Some example of scanf( ) are as :-

```
# Include < stdio.h>
    main ( )
    {
    Int. marks ;
    ---------
    Scanf ("%d", & marks);
    }
```

The control string contains only one conversion specification character "%d" which implies that an integer value should be entered as input. The entered value will be stored in the variable marks.

Example 2:-

```
# Include < stdio.h>
    main ( )
    {
    Char str [30] ;
    ---------
    Scanf ("%s", & str);
    ---------
    }
```

Example 3:-

```
# Include < stdio.h>
    main ( )
    {
```

PAGE:
DATE: / /

```
int basic;
float hra;
Char grade;
Scanf("%d%f %c", &basic, &hra, &grade);
}
```

✡. Writting output Data :-

output data can be written from computer memory to standard output device using printf library function. C provides the printf( ) to display the data on the monitor. With this function all type of values (Numeric, char, or String) can be written as output.

✓ The printf( ) is including in stdio.h
The printf( ) can be written as :-

```
printf("control String", Variable, Variable);
```

In this function the control string contains conversion specification character and text it. It should be enclosed within double quotes. The name of variable should no be proced by an ampersand (&) sign.

PAGE:
DATE: / /

☆. Some examples of printf( ) are as:-

1.
```
# include <stdio.h>
    main ( ).
    {
printf("c is excellent \n");
    }
```

Output = c is excellent.
The control string has only text and no conversion specification character. Hence the output is only text.

2.
```
# include <stdio.h>
    main ( ).
    {

    int basic = 2000;
    - - - - - - - - - - - -
printf("%d", basic);
    }
```

Output = 2000.

3.
```
# include <stdio.h>.
    main ( )
    {
    char ch = '$';
```

DATE: / /

printf("%c", ch);

}

☆. getch( ) :- getch is used to get a character from consoles. It can be used to hold program execution. Its primary purpose which is to wait until the users enter a character.

☆. putchar( ) :-
Putchar( ) out one character at a time to the standard output. Putchar( ) is used to write a character on standard output screen. In a C program we can use putchar( ) as below :-
putchar(char);
→ Character is a variable or value.

☆. getchar( ) :- getchar( ) is used to get / read a character from keyboard input. In a C program we can use getchar( ) as below :-
getchar(char);

```
# include <stdio.h>
main ( )
{
char ch;
Printf ("enter a character");
ch = get char ( );
printf ("The entered character is");
Put char (ch);
}
```

✡. <u>Clrscr</u> :-
Clrscr means clean Screen. It will clear the output of the previous program.

✡. <u>Operators</u> :- An operator specify an operation to be perform. C include a large no. of operators that fall under several different categories which are as :-

C operators

- UNARY operators
- BINARY operators
- TERNARY operators.

www.dreamstudy.tk

DATE: / /

1. **UNARY Operators :-**
An operator that act upon any on operand is known as UNARY operator.

☆. **UNARY Minus :-**
Any possible positive operand associated with UNARY MINUS operator gets its value changes to negative.

for ex :- $a = 3.$
$b = 4.$
$c = a + (-b)$
$c = 3 - 4 = -1$

2. **BINARY Operator :-**

- Arithmetic op^r
- logical op^r
- Relational op^r
- Bitwise op^r.

2. **BINARY op^r :-**

(i) **Arithmatic op^r :-** Arithmetic op^r are used to perform the basic arithmatic ~~operator~~ operations. Such as addition, Subtraction, Multiplication, and division.

operator	Purpose
+	Addition
−	Subtraction
X	Multiplication
/	division

DATE: / /

(ii) **Relational opr :-** These are used to compare two operands. They define the relationship existing between two constants or variables.
They result in either a true or False value. The value of true is non zero (1) and the value of False is (0).

Operator	Meaning
<	is less than
≤	is less than or equal to
>	is greater than
≥	is greater than or equal to
==	equal to
!=	is not equal to

(iii) **Logical operators :-**
This class of C operators is used to take decisions these are three such operators they are
AND
OR
NOT
The first two are binary operator and the last one is unary operator. The result of these operations either true or false.

Operator	Meaning	Precedence	Associativity
&& (Double Ampersand)	Logical AND	2	Left to Right
|| (Double pipline)	Logical OR	3	"
! Enclamation	Logical NOT	1	"

(iv) <u>Bitwise operator</u> :-

All data items are stored in the Computer memory as a sequence of bits (0, 1). C has the ability to Support the manuplation of data at the bit level. Bitwise operator are used for operations on individual bits, Bitwise operator, oprande on integer only. The bitwise op^r are as:-

Bitwise operator	Meaning.
&	Bitwise AND (Ampersand)
|	Bitwise OR (Pipeline).
~	One's compliment (Tilde)
<<	Left Shift
>>	Right Shift
^	Bitwise XOR (CARET).

PAGE:
DATE: / /

Imp 3. <u>TERNARY operator</u> :- (conditional opr):-
The Ternary opr is a conditional opr in C. There is the only one opr in C. It takes three operands. The Symbole "?" is use as ternary operator in C.

<expression>?<value 1>:<value 2>;

→ Relational Expression.

Example:-

C=a<b?a:b;

Here C will be sign the value of a if a is less than b otherwise it will assign the value of b.